

Smytten E-Commerce Database Management System

CHAPTER – 6

Seller and Inventory Management

1. Introduction

The Smytten E-Commerce Database Management System is designed to manage products, sellers, and inventory information efficiently. The Seller table stores details about sellers who provide products through the platform. The Inventory table maintains information about the available stock of each product.

The system establishes relationships between sellers, products, and inventory using primary and foreign keys. It also helps identify whether products are available or unavailable based on their stock quantity. Inventory status reports can be generated to monitor product availability and stock levels.

2. Create Seller and Inventory Tables

2.1 Seller Table

The SELLER table stores information about sellers associated with the Smytten platform.

Attribute	Description	Constraint
SELLER_ID	Unique identification of seller	Primary Key
SELLER_NAME	Name of seller	NOT NULL
EMAIL	Seller email address	UNIQUE
PHONE_NUMBER	Seller contact number	—
ADDRESS	Seller location/address	—

SQL Code:

```
SQL> CREATE TABLE SELLER (  
2     SELLER_ID NUMBER PRIMARY KEY,  
3     SELLER_NAME VARCHAR2(100) NOT NULL,  
4     EMAIL VARCHAR2(100) UNIQUE,  
5     PHONE_NUMBER VARCHAR2(15),  
6     ADDRESS VARCHAR2(200)  
7 );
```

Table created.

Insert 5 Seller Records:

```
SQL> INSERT INTO SELLER
  2 VALUES (1, 'Mamaearth', 'mamaearth@gmail.com', '9876500001', 'Gurugram');

1 row created.

SQL>
SQL> INSERT INTO SELLER
  2 VALUES (2, 'Minimalist', 'minimalist@gmail.com', '9876500002', 'New Delhi');

1 row created.

SQL>
SQL> INSERT INTO SELLER
  2 VALUES (3, 'Dot & Key', 'dotandkey@gmail.com', '9876500003', 'Kolkata');
Enter value for key:
old 2: VALUES (3, 'Dot & Key', 'dotandkey@gmail.com', '9876500003', 'Kolkata')
new 2: VALUES (3, 'Dot ', 'dotandkey@gmail.com', '9876500003', 'Kolkata')

1 row created.

SQL> INSERT INTO SELLER
  2 VALUES (4, 'Plum', 'plum@gmail.com', '9876500004', 'Mumbai');

1 row created.

SQL>
SQL> INSERT INTO SELLER
  2 VALUES (5, 'WOW Skin Science', 'wowskinscience@gmail.com', '9876500005', 'Bangalore');

1 row created.
```

2.2 Inventory Table

The INVENTORY table stores stock information for products and tracks their availability.

Attribute	Description	Constraint
INVENTORY_ID	Unique inventory identification	Primary Key
PRODUCT_ID	Identifies the product	Foreign Key
AVAILABLE_STOCK	Quantity currently available	NOT NULL
LAST_UPDATED	Date of latest stock update	DEFAULT
PRODUCT_STATUS	Availability of product	—

SQL Code:

```
SQL> CREATE TABLE INVENTORY (
  2     INVENTORY_ID NUMBER PRIMARY KEY,
  3     PRODUCT_ID NUMBER NOT NULL,
  4     AVAILABLE_STOCK NUMBER NOT NULL CHECK (AVAILABLE_STOCK >= 0),
  5     LAST_UPDATED DATE DEFAULT SYSDATE,
  6     PRODUCT_STATUS VARCHAR2(20),
  7     CONSTRAINT FK_INVENTORY_PRODUCT
  8     FOREIGN KEY (PRODUCT_ID)
  9     REFERENCES PRODUCT2(PRODUCT_ID)
 10 );
```

Table created.

Insert 5 Inventory Records:

```
SQL> INSERT INTO INVENTORY
  2 VALUES (1, 101, 50, SYSDATE, 'Available');

1 row created.

SQL>
SQL> INSERT INTO INVENTORY
  2 VALUES (2, 102, 40, SYSDATE, 'Available');

1 row created.

SQL>
SQL> INSERT INTO INVENTORY
  2 VALUES (3, 103, 35, SYSDATE, 'Available');

1 row created.

SQL>
SQL> INSERT INTO INVENTORY
  2 VALUES (4, 104, 0, SYSDATE, 'Unavailable');

1 row created.

SQL>
SQL> INSERT INTO INVENTORY
  2 VALUES (5, 105, 45, SYSDATE, 'Available');

1 row created.

SQL>
SQL> COMMIT;

Commit complete.
```

3. Establish Relationships Between Sellers, Products and Stock

The database establishes relationships using primary and foreign keys.

SELLER → PRODUCT2: One seller can provide multiple products.

PRODUCT2 → INVENTORY: Products are connected to their inventory records.

PRODUCT_ID in INVENTORY is a foreign key referencing PRODUCT_ID in PRODUCT2.

Relationship Structure:

SELLER
|
| Supplies
↓
PRODUCT2
|
| Has Stock
↓
INVENTORY

4. Maintain Seller Product Information

Seller information can be maintained along with product information by adding SELLER_ID to PRODUCT2 and establishing a foreign key relationship.

```
SQL> ALTER TABLE PRODUCT2
  2  ADD SELLER_ID NUMBER;

Table altered.

SQL> ALTER TABLE PRODUCT2
  2  ADD CONSTRAINT FK_PRODUCT_SELLER
  3  FOREIGN KEY (SELLER_ID)
  4  REFERENCES SELLER(SELLER_ID);

Table altered.
```

Assign Sellers to Products:

```
SQL> UPDATE PRODUCT2
  2  SET SELLER_ID = 1
  3  WHERE PRODUCT_ID IN (101, 102);

2 rows updated.

SQL>
SQL> UPDATE PRODUCT2
  2  SET SELLER_ID = 2
  3  WHERE PRODUCT_ID IN (103, 104);

2 rows updated.

SQL>
SQL> UPDATE PRODUCT2
  2  SET SELLER_ID = 3
  3  WHERE PRODUCT_ID = 105;

1 row updated.
```

Display Seller Product Information:

```
SQL> SELECT
  2     S.SELLER_ID,
  3     S.SELLER_NAME,
  4     P.PRODUCT_ID,
  5     P.PRODUCT_NAME,
  6     P.PRICE
  7 FROM SELLER S
  8 JOIN PRODUCT2 P
  9 ON S.SELLER_ID = P.SELLER_ID
 10 ORDER BY S.SELLER_ID;
```

```
SELLER_ID
-----
SELLER_NAME
-----
PRODUCT_ID
-----
PRODUCT_NAME
-----
PRICE
-----
1
Mamaearth
101

SELLER_ID
-----
SELLER_NAME
-----
PRODUCT_ID
-----
PRODUCT_NAME
-----
PRICE
-----
Face Wash
349
```

5. Track Available and Unavailable Products:

A product is Available when its stock is greater than zero. If the stock is zero, it is Unavailable.

```
SQL> SELECT
  2     I.INVENTORY_ID,
  3     P.PRODUCT_ID,
  4     P.PRODUCT_NAME,
  5     I.AVAILABLE_STOCK,
  6     CASE
  7         WHEN I.AVAILABLE_STOCK > 0 THEN 'Available'
  8         ELSE 'Unavailable'
  9     END AS PRODUCT_STATUS
 10 FROM INVENTORY I
 11 JOIN PRODUCT2 P
 12 ON I.PRODUCT_ID = P.PRODUCT_ID;
```

```

INVENTORY_ID PRODUCT_ID
-----
PRODUCT_NAME
-----
AVAILABLE_STOCK PRODUCT_STA
-----
          1          101
Face Wash          50 Available

          2          102
Moisturizer        40 Available

INVENTORY_ID PRODUCT_ID
-----
PRODUCT_NAME
-----
AVAILABLE_STOCK PRODUCT_STA
-----

          3          103
Lipstick           35 Available

          4          104
Mascara

```

INVENTORY_ID	PRODUCT_ID	PRODUCT_NAME	AVAILABLE_STOCK	PRODUCT_STATUS
1	101	Face Wash	50	Available
2	102	Moisturizer	40	Available
3	103	Lipstick	35	Available
4	104	Mascara	0	Unavailable
5	105	Shampoo	45	Available

```

INVENTORY_ID PRODUCT_ID
-----
PRODUCT_NAME
-----
AVAILABLE_STOCK PRODUCT_STA
-----
          0 Unavailable

          5          105
Shampoo           45 Available

```

6. Generate Inventory Status Reports:

The following query generates an inventory status report showing seller, product, stock, and availability.

```
SQL> SELECT
  2     S.SELLER_NAME,
  3     P.PRODUCT_NAME,
  4     I.AVAILABLE_STOCK,
  5     CASE
  6         WHEN I.AVAILABLE_STOCK > 0 THEN 'Available'
  7         ELSE 'Unavailable'
  8     END AS PRODUCT_STATUS
  9 FROM INVENTORY I
 10 JOIN PRODUCT2 P
 11 ON I.PRODUCT_ID = P.PRODUCT_ID
 12 LEFT JOIN SELLER S
 13 ON P.SELLER_ID = S.SELLER_ID
 14 ORDER BY S.SELLER_NAME;
```

```
SELLER_NAME
-----
PRODUCT_NAME
-----
AVAILABLE_STOCK  PRODUCT_STA
-----
Dot
Shampoo
                45 Available

Mamaearth
Face Wash
                50 Available
```

```
SELLER_NAME
-----
PRODUCT_NAME
-----
AVAILABLE_STOCK  PRODUCT_STA
-----

Mamaearth
Moisturizer
                40 Available

Minimalist
Mascara

SELLER_NAME
-----
PRODUCT_NAME
-----
AVAILABLE_STOCK  PRODUCT_STA
-----
                0 Unavailable

Minimalist
Lipstick
                35 Available
```

7. PL/SQL Program for Inventory Status Report:

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
2
3     CURSOR Inventory_Report IS
4     SELECT
5         S.SELLER_NAME,
6         P.PRODUCT_NAME,
7         I.AVAILABLE_STOCK,
8         CASE
9             WHEN I.AVAILABLE_STOCK > 0
10            THEN 'Available'
11            ELSE 'Unavailable'
12        END AS PRODUCT_STATUS
13     FROM INVENTORY I
14     JOIN PRODUCT2 P
15     ON I.PRODUCT_ID = P.PRODUCT_ID
16     LEFT JOIN SELLER S
17     ON P.SELLER_ID = S.SELLER_ID
18     ORDER BY S.SELLER_NAME;
19
20 BEGIN
21
22     DBMS_OUTPUT.PUT_LINE('=====');
23     DBMS_OUTPUT.PUT_LINE('          INVENTORY STATUS REPORT');
24     DBMS_OUTPUT.PUT_LINE('=====');
25
26     FOR REC IN Inventory_Report LOOP
27
28         DBMS_OUTPUT.PUT_LINE(
29             'Seller: ' || REC.SELLER_NAME ||
30             ' | Product: ' || REC.PRODUCT_NAME ||
31             ' | Stock: ' || REC.AVAILABLE_STOCK ||
32             ' | Status: ' || REC.PRODUCT_STATUS
33         );
34
35     END LOOP;
36
37     DBMS_OUTPUT.PUT_LINE('=====');
38     DBMS_OUTPUT.PUT_LINE('Report generated successfully.');
```

```
=====
INVENTORY STATUS REPORT
=====
Seller: Dot | Product: Shampoo | Stock: 45 | Status: Available
Seller: Mamaearth | Product: Face Wash | Stock: 50 | Status: Available
Seller: Mamaearth | Product: Moisturizer | Stock: 40 | Status: Available
Seller: Minimalist | Product: Mascara | Stock: 0 | Status: Unavailable
Seller: Minimalist | Product: Lipstick | Stock: 35 | Status: Available
=====
Report generated successfully.

PL/SQL procedure successfully completed.
```

8. Conclusion:

The Seller and Inventory Management System provides an organized method for maintaining seller, product, and stock information in the Smytten E-Commerce Database. The SELLER table stores seller details, while the INVENTORY table tracks product stock and availability. Primary and foreign keys establish relationships between the tables. SQL JOIN and PL/SQL operations help maintain seller-product information and generate inventory status reports.